axiateam
RDT company

Anexo 5A

SSR: Frameworks para Rendering en Backend

Curso: TypeScript

90 min

Objetivos de Aprendizaje

- Comprender que Webpack NO hace SSR - solo empaqueta
- Conocer los frameworks modernos para SSR: Next.js, Remix, Astro
- Evaluar cuando migrar de Webpack SPA a SSR
- Aplicar el patrón correcto según el tipo de aplicación
- Migrar de SPA Webpack a Next.js SSR
- Tipar Server Components con params y searchParams
- Implementar Data Fetching con tipos y Zod
- Usar Streaming y Suspense con tipos
- Configurar Metadata API para SEO
- Crear API Routes avanzados con validación

1. Contexto: El problema del cliente

10 minutos

Escenario: "Se renderiza en el back"

El cliente dice esto pero usualmente significa que usa Webpack, que solo empaqueta JS. SSR real necesita un framework que ejecute React en el servidor.

La confusión común:

- "Uso Webpack" → pero Webpack solo empaqueta, NO hace SSR
- "Quiero renderizar en backend" → necesitas un FRAMEWORK

Lo que el cliente ACTUALMENTE tiene:

Webpack → empaqueta JS → navegador → React hydrate → render (NO es SSR)

Webpack != SSR

Webpack es un bundler, no un framework. Solo empaqueta archivos, no ejecuta React en servidor.

Herramienta	Tipo	Hace SSR?
Webpack	Bundler	NO
Vite	Dev server + Bundler	NO
Next.js	Framework	SI
Remix	Framework	SI
Astro	Framework	SI

2. Webpack != SSR

10 minutos

Cuándo Migrar a Next.js

Señales claras para migrar: SEO importante, contenido dinámico frecuente, primer paint lento.

Señal	Acción
Contenido cambia frecuentemente	Migrar a SSR
SEO es importante	Migrar a SSR
Primer paint lenta	Migrar a SSR
App muy interactiva (dashboards)	Quedarse en SPA

Beneficios: SEO mejorado, FCP más rápido, mejor UX

3. Next.js

15 minutos

¿Por qué Next.js?

Framework más popular con App Router, API routes integradas, y deploy zero-config con Vercel.

- Más popular (20M+ descargas/semana)
- Usa Webpack internamente (o Turbopack)
- Vercel detrás
- App Router (React Server Components)
- API Routes integradas

ROI: Casos donde Next.js brilla

E-commerce, blogs, y landing pages se benefician más por SEO y velocidad.

Caso	Por qué Next.js
E-commerce	SEO crítico, contenido dinámico, metadata dinámica
Blogs/Noticias	SSG + ISR para velocidad + contenido frecuente
Dashboards	Client components para interactividad
Landing pages	SSG instantáneo, metadata automática

ROI: Deploy zero-config con Vercel, Netlify. curva de aprendizaje mínima (ya sabes React).

Crear proyecto Next.js

Comando `npx create-next-app` configura todo: TypeScript, App Router, y estructura de carpetas.

```
npx create-next-app@latest my-app --typescript
```

Lab: Crear proyecto Next.js básico

```
# Crear proyecto Next.js con TypeScript
npx create-next-app@latest tareas-ssr --typescript --app --src-dir --import-alias "@/*"

cd tareas-ssr
npm run dev
# http://localhost:3000
```

4. Server vs Client Components

15 minutos

La regla de oro

Server Components por defecto. Solo usar 'use client' cuando necesites hooks o eventos.

En Next.js App Router, los componentes son Server Components por defecto.

Característica	Server	Client
Renderizado	Servidor	Cliente
Hooks (useState, useEffect)	No	Sí
Event handlers	No	Sí
Acceso BD/FS	Sí	No

Árbol de decisiones

Pregúntate: ¿tiene interactividad? ¿accede a BD/FS? Responde las preguntas para saber qué tipo de componente usar.

```
¿Tiene interactividad? (useState, onClick)
  → SI → Client Component ('use client')
  → NO → ¿Accede a BD/FS?
      → SI → Server Component
      → NO → Server Component (por defecto)
```

Server vs Client: Beneficios Concretos

Server: SEO automático, acceso directo a BD, 0KB JS. Client: interactividad completa.

Aspecto	Server	Client
SEO	✅ Automatico	❌ Requiere cliente
Acceso BD/FS	✅ Directo	❌ Via API
Bundle size	✅ 0 KB JS	❌ Todo el codigo
Interactividad	❌ No	✅ Completa
Time to interactive	✅ Inmediato	❌ Espera JS

Tip: Usa Server por defecto. Solo cambia a Client cuando NECESITES hooks o eventos.

Lab: Migrar componente a Server Component

```
// ✅ Server Component (sin interactividad)
// app/components/TaskList.tsx
export default async function TaskList() {
  const tasks = await db.query('SELECT * FROM tasks');
  return (
    <ul>
      {tasks.map(task => <li key={task.id}>{task.title}</li>)}
    </ul>
  );
}

// ❌ Client Component (con useState)
// app/components/Counter.tsx
'use client';
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
```

5. Routing y API Routes

15 minutos

Routing: app/tasks/[id]/page.tsx

El App Router usa archivos para definir rutas. Los parámetros vienen en `params` (promesa en Next.js 15+).

```
interface PageParams {
  params: { id: string };
}

export default async function TaskPage({ params }: PageParams) {
  const task = await getTask(params.id);
  return <TaskDetail task={task} />;
}
```

Route Handlers: app/api/tasks/route.ts

API Routes permiten crear endpoints REST directamente en Next.js.

```
export async function GET() {
  const tasks = await getTasks();
  return NextResponse.json(tasks);
}

export async function POST(request: NextRequest) {
  const body = await request.json();
  const created = await createTask(body);
  return NextResponse.json(created, { status: 201 });
}
```

Pages vs API Routes: Cuándo usar cada uno

Pages para UI (SEO). API Routes para lógica (CRUD, auth, webhooks).

Aspecto	Pages	API Routes
Proposito	Renderizar UI	Manejar requests
Returns	HTML/JSX	JSON
Browser	Directo	Via fetch
SEO	Indexable	No indexable

Regla: Pages para UI. API Routes para logic (CRUD, auth, webhooks).

Lab: Crear rutas y API en Next.js

```
// app/tasks/page.tsx - Lista de tareas
export default async function TasksPage() {
  const tasks = await fetch('http://localhost:3000/api/tasks').then(r => r.json());
  return <div>{tasks.map(t => <TaskItem key={t.id} task={t} />)}</div>;
}

// app/tasks/[id]/page.tsx - Detalle de tarea
interface PageProps { params: Promise<{ id: string }>; }
export default async function TaskPage({ params }: PageProps) {
  const { id } = await params;
  const task = await fetch(`/api/tasks/${id}`).then(r => r.json());
  return <TaskDetail task={task} />;
}

// app/api/tasks/route.ts - API GET/POST
export async function GET() {
  const tasks = await db.getTasks();
  return NextResponse.json(tasks);
}
```

6. Tipado de Server Components

15 minutos

Params como Promise (Next.js 15+)

En Next.js 15+, `params` es una Promise que debe resolverse con `await`.

```
interface PageProps {
  params: Promise<{ id: string }>;
}

export default async function TaskPage({ params }: PageProps) {
  const { id } = await params;
  const task = await getTask(id);
  return <TaskDetail task={task} />;
}
```

SearchParams tipado

searchParams también es Promise. Tipalo para evitar errores en runtime.

```
interface PageSearchParams {
  searchParams: Promise<{ page?: string; filter?: string }>;
}

export default async function TasksPage({ searchParams }: PageSearchParams) {
  const { page, filter } = await searchParams;
  // ...
}
```

Tipado: Por qué importa en Server Components

Los tipos dan seguridad en refactoring, autocompletado en el IDE, y errores en build no en producción.

Sin tipos	Con tipos
params.id puede ser string | undefined	params.id es string
task.title puede fallar en runtime	task.title es string
Refactoring peligroso	Refactoring seguro
searchParams sin autocomplete	searchParams con autocomplete

Beneficio: Errores en build, no en production. DX mejorado con autocomplete.

Lab: Tipar Server Components

```
// app/tasks/[id]/page.tsx - Tipado completo
import { getTask } from '@/lib/tasks';

interface PageProps {
  params: Promise<{ id: string }>;
}

export default async function TaskPage({ params }: PageProps) {
  const { id } = await params;
  const task = await getTask(id);

  if (!task) notFound();

  return (
    <div>
      <h1>{task.title}</h1>
      <p>{task.description}</p>
    </div>
  );
```

7. Data Fetching con Tipos

15 minutos

Fetch tipado + Zod

Un solo schema Zod define tipos y validación. Útil para compartir tipos entre frontend y backend.

```
import { z } from 'zod';

const TaskSchema = z.object({
  id: z.number(),
  title: z.string(),
  status: z.enum(['pending', 'completed']),
});

type Task = z.infer<typeof TaskSchema>;

async function getTasks(): Promise<Task[]> {
  const res = await fetch('/api/tasks');
  return res.json();
}
```

Beneficio: Un solo schema = tipos en frontend y backend

Fetch + Zod: Por qué mejor que any

```
// ❌ Con any - sin validación, sin tipos
const data: any = await fetch('/api/tasks').then(r => r.json());
data.title.toUpperCase(); // Puede fallar en runtime!

// ✅ Con Zod - validación + tipos
const data = await fetch('/api/tasks')
  .then(r => r.json())
  .then(data => TaskSchema.array().parse(data));
data[0].title.toUpperCase(); // Type-safe!
```

Aspecto	any	Zod
Validación	❌ No	✅ Sí
Errores	Runtime	Build time
Autocomplete	❌ No	✅ Sí

Lab: Data Fetching Tipado con Zod

```
// types/task.ts
import { z } from 'zod';

export const TaskSchema = z.object({
  id: z.number(),
  title: z.string().min(1).max(100),
  status: z.enum(['pending', 'inProgress', 'completed']),
  priority: z.enum(['low', 'medium', 'high']),
});

export type Task = z.infer<typeof TaskSchema>;
export type CreateTaskInput = z.infer<typeof CreateTaskSchema>;

// lib/api.ts
export async function getTasks() {
  const res = await fetch('/api/tasks');
  const data = await res.json();
  return TaskSchema.array().parse(data); // throw if invalid
}
```

8. Streaming y Suspense

15 minutos

Archivos especiales

Next.js reconoce estos archivos automáticamente: loading.tsx, error.tsx, not-found.tsx.

Archivo	Cuando se muestra
loading.tsx	Mientras carga la página
error.tsx	Cuando hay error
not-found.tsx	Cuando notFound() es llamado

Suspense

Suspense permite mostrar contenido de fallback mientras los componentes asíncronos cargan.

```
import { Suspense } from 'react';

export default function Page() {
  return (
    <Suspense fallback={<Skeleton />}>
      <TaskList />
    </Suspense>
  );
}
```

Streaming: Mejora UX en aplicaciones reales

El contenido aparece progresivamente en lugar de esperar a que todo cargue. Reduce la tasa de rebote.

Sin Streaming	Con Streaming
Pagina en blanco hasta que TODO cargue	Contenido aparece progresivamente
Usuario espera 3-5s sin feedback	Skeleton muestra progreso
Alta tasa de rebote	Usuario ve contenido parcial antes

Caso real: E-commerce con 100 productos. Sin streaming: 5s de espera. Con streaming: productos aparecen en 500ms.

Lab: Implementar Streaming y Suspense

```
// app/tasks/loading.tsx
export default function Loading() {
  return <div className="skeleton">{Array(5).fill(<div className="skeleton-row" />)}</div>;
}

// app/tasks/error.tsx
'use client';
export default function Error({ error }: { error: Error }) {
  return <div>Error: {error.message}</div>;
}

// app/tasks/not-found.tsx
export default function NotFound() {
  return <div>Task not found</div>;
}

// app/tasks/page.tsx - con Suspense
import { Suspense } from 'react';
```

9. Metadata API

10 minutos

Metadata estática

```
import type { Metadata } from 'next';

export const metadata: Metadata = {
  title: 'Task Manager',
  description: 'Lista completa de tareas',
  openGraph: {
    title: 'Task Manager',
    type: 'website',
  },
};
```

Metadata dinámica

generateMetadata permite crear títulos y descripciones únicos para cada página dinámicamente.

```
export async function generateMetadata({ params }: Props): Promise<Metadata> {
  const { id } = await params;
  const task = await getTask(id);

  return {
    title: task ? `${task.title} - Task Manager` : 'Not found',
  };
}
```

Metadata API: Impacto en SEO y Compartición

Sin Metadata	Con Metadata
Google muestra "untitled" o URL	Google muestra título descriptivo
No hay descripción en resultados	Descripción personalizada
Links no tienen preview	OpenGraph con imagen y descripción
Redes sociales muestran URL	Cards con imagen y título

ROI: +30% CTR en resultados de búsqueda con metadata rica.

Lab: Metadata API con Tipos

```
// app/layout.tsx - Metadata global
import type { Metadata } from 'next';

export const metadata: Metadata = {
  title: 'Task Manager SSR',
  description: 'App de tareas con Server Side Rendering',
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return <html><body>{children}</body></html>;
}

// app/tasks/[id]/page.tsx - Metadata dinámica
interface Props { params: Promise<{ id: string }>; }

export async function generateMetadata({ params }: Props): Promise<Metadata> {
  const { id } = await params;
  const task = await getTask(id);
```

10. API Routes Avanzados

15 minutos

Headers, Cookies y Query Params

Las API Routes acceden a headers, cookies y query params a través del objeto NextRequest.

```
export async function GET(request: NextRequest) {
  const authHeader = request.headers.get('authorization');
  const sessionCookie = request.cookies.get('session');
  const { searchParams } = new URL(request.url);
  const page = searchParams.get('page');

  return NextResponse.json({ auth, session, page });
}
```

API Routes vs Backend Externo: Cuándo elegir

Aspecto	API Routes (Next.js)	Backend Externo
Deploy	Zero-config con Next.js	Necesita servidor propio
Microservicios	Uno por funcionalidad	Arquitectura distribuida
Escalabilidad	Escala con Next.js	Escala independientemente
Casos simples	✅ CRUD rápido	❌ Overkill
Casos complejos	❌ Limitado	✅ Flexible

Regla: Empieza con API Routes. Migra a backend externo cuando la lógica sea muy compleja o necesites servicios específicos.

Lab: API Routes Completos

```
// app/api/tasks/route.ts
export async function GET(request: NextRequest) {
  const { searchParams } = new URL(request.url);
  const status = searchParams.get('status');
  const page = Number(searchParams.get('page') || 1);

  const tasks = await getTasks({ status, page });
  return NextResponse.json(tasks);
}

export async function POST(request: NextRequest) {
  const body = await request.json();
  const result = CreateTaskSchema.safeParse(body);

  if (!result.success) {
    return NextResponse.json({ error: result.error }, { status: 400 });
  }

  const task = await createTask(result.data);
```

11. Ejercicio Integrador

Migrar Caso 2r (Biblioteca) a Next.js SSR

Requisitos

- Crear proyecto Next.js con la app
- Convertir componentes existentes a Server/Client Components
- Migrar routing a estructura Next.js App Router
- Crear páginas necesarias
- Implementar API routes para CRUD
- Configurar variables de entorno

12. Resumen

5 minutos

Puntos Clave

1. Webpack NO hace SSR - solo empaqueta
2. SSR = Framework: Next.js, Remix, Astro
3. Server vs Client Components - Server por defecto, use client para interactividad
4. Routing - estructura de carpetas en app/
5. API Routes - crear endpoints sin backend separado
6. Tipado de Server Components - params como Promise, searchParams
7. Data Fetching - fetch tipado, Zod para validación
8. Streaming - Suspense, loading.tsx, error.tsx, not-found.tsx
9. Metadata API - SEO con tipos estáticos y dinámicos
10. API Routes avanzados - headers, cookies, query params con tipos

13. Referencias

Recursos Adicionales

- Next.js Documentation
- Next.js App Router
- React Server Components
- SSR vs SSG
- Next.js Metadata API
- Zod Validation

Gracias!

Anexo 5A Completado

Siguiente: Anexo 8A - Patrones TS Avanzados